

UNIVERSITAT POLITÈCNICA DE CATALUNYA

Escola Tècnica Superior d'Enginyeria de Telecomunicacions de Barcelona (ETSETB)

Practice 2

Grau en Enginyeria Física

Authors: Arnau Torret & Arnau Montull

Subject: SIMCOM

Date: October 23, 2026

1 Exercise 1

1.1 Question

Simulation of a simple Lennard-Jones fluid, such as liquid argon, with a hand-made FORTRAN MD code which incorporates the Leap-frog Verlet algorithm.

STEP 1: Understanding the code.

- Open the FORTRAN file `leapfroglj.f` and have a look.
- Note that you'll need 2 auxiliary files: (1) `leap-lj.data` and (2) `leap-conf.data`, containing (1) data input and (2) initial coordinates and velocities, respectively.
- The main program is ready to simulate a LJ liquid of 500 atoms of mass 39.948 uma, $\sigma = 3.405$ Å and $\varepsilon/k_B = 119.8$ K (Argon), with a time step of $\Delta t = 0.01$ ps. Remember that the LJ potential is of the form:

$$V_{\mathrm{LJ}}(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]. \tag{1}$$

- Compile the code and make a run of 10000 time steps. If you compute the temperature T and total energy E (kinetic + potential) you will get the following figures.

1.2 Code

This step runs the leap-frog program supplied with the practice, transcribed to free-form Fortran without algorithmic changes; only the accumulation and printing of the averages was added. The run uses the parameters of the statement (10000 steps, $N = 500$). The full program, with every later addition, is listed in the Anex.

```
   sumt = 0.d0
   sumt2 = 0.d0
   sume = 0.d0
   sume2 = 0.d0

   ! 5. Start the loop to generate new configurations
   do i = 1, nconf
      call forces(natoms, r, boxlength, accel, rc, epot)
      call velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, &
                  boxlength)
      etot = ecin + epot
      write(3,*) i*deltat, etot
      write(4,*) i*deltat, temp
      sumt = sumt + temp
      sumt2 = sumt2 + temp*temp
      sume = sume + etot
      sume2 = sume2 + etot*etot
   end do
   close(3)
   close(4)

   ! Mean and standard deviation of T and E_tot over the run
   tmean = sumt/nconf
   emean = sume/nconf
   tsd = dsqrt(sumt2/nconf - tmean**2)
   esd = dsqrt(sume2/nconf - emean**2)
   print *, 'steps  = ', nconf
   print *, '<T>    = ', tmean, '  sigma_T = ', tsd
   print *, '<Etot> = ', emean, '  sigma_E = ', esd
```

Listing 1: Main loop with the running sums of T and E_{exttot} (added) and the printed means and standard deviations.

1.3 Results

With $L^* = 8.427$ ($\rho^* = 0.835$) and $\Delta t^* = 4.637 \times 10^{-3}$ ($\tau = 2.156$ ps), the 10000 steps cover $t^* = 46.37$ (100 ps). Figure 1 shows T and $E_{\mathrm{tot}} = E_{\mathrm{kin}} + E_{\mathrm{pot}}$.

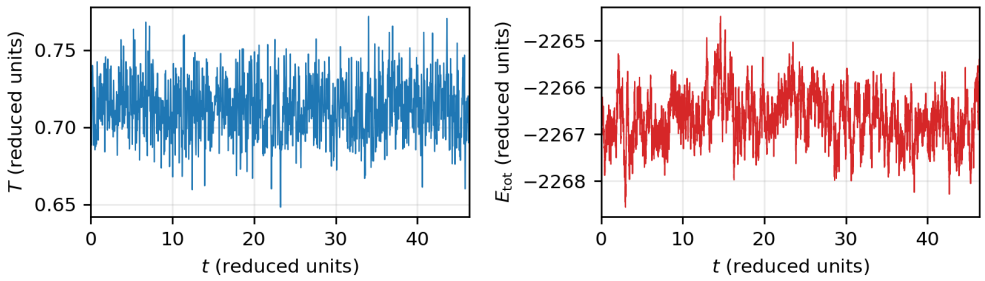


Figure 1: Temperature $T = 2E_{\mathrm{kin}}/(3N - 3)$ (left) and total truncated energy E_{tot} (right), in reduced units.

1.4 Discussion & Notes

- **Energy:** leap-frog Verlet is symplectic, so E_{tot} should only oscillate. We get $\langle E_{\mathrm{tot}} \rangle = -2266.65$, and $\sigma_E = 0.537$ (0.024%), a very small fluctuation.
- **Temperature:** $\langle T^* \rangle = 0.713$ (85.4 K), $\sigma_T = 0.0175$ (2.5%). That is a much larger relative fluctuation than for E_{tot}, as E_{kin} and E_{pot} are exchanged. Both curves match the reference figures for a dense liquid.

2 Exercise 2

2.1 Question

STEP 2: Analysing the structure and calculating thermodynamic properties.

- Modify the code to calculate the radial distribution function (RDF) $g(r)$ from the MD simulation. Compute the RDF for a run of 10000 steps. Plot $g(r)$ vs. r using reduced units.
- Modify the code to calculate the pressure P. Compute the mean values of the total and different contributions to E and P. Include the tail corrections. Analyze the convergence of the results as a function of the simulation length. Use reduced units.

2.2 Code

Only the lines added for the radial distribution function, the virial pressure and the long-range tails are shown. The full program is listed in the Anex.

```
9     double precision mass, nid
10
11    ! 1. Defining dimensions
12    dimension r(3,1000), vinf(3,1000), accel(3,1000)
13    parameter (nhis = 1000)
14    dimension g(nhis)
```

Listing 2: Histogram storage. `nid` is real because a name starting with `n` would be an integer under implicit typing.

```
49    pi = 4.d0*datan(1.d0)
50    delg = boxlength/(2.d0*dfloat(nhis))
51    rho = dfloat(natoms)/boxlength**3
52    etail = (8.d0*pi/3.d0)*rho*dfloat(natoms)* &
53            (1.d0/(3.d0*rc**9) - 1.d0/rc**3)
54    ptail = (16.d0*pi/3.d0)*rho**2* &
55            (2.d0/(3.d0*rc**9) - 1.d0/rc**3)
56    do j = 1, nhis
57       g(j) = 0.d0
58    end do
59    ekin_sum = 0.d0
60    epot_sum = 0.d0
61    pkin_sum = 0.d0
62    pvir_sum = 0.d0
```

Listing 3: Tail corrections and zeroing of the RDF and pressure accumulators before the loop.

```
74        ! Conserved NVE energy (no tail). Thermodynamic totals include it.
75        etot = ecin + epot
76        etot_corr = etot + etail
77        pkin = 2.d0*ecin/(3.d0*boxlength**3)
78        pvir = vir/(3.d0*boxlength**3)
79        ptot = pkin + pvir + ptail
80        ekin_sum = ekin_sum + ecin
81        epot_sum = epot_sum + epot
82        pkin_sum = pkin_sum + pkin
83        pvir_sum = pvir_sum + pvir
84        write(3,*) i*deltat, etot
85        write(4,*) i*deltat, temp
86        write(8,*) i*deltat, etot_corr, pkin, pvir, ptail, ptot
```

Listing 4: Energy and pressure contributions accumulated on every step of the MD loop.

```
92    ekin_avg = ekin_sum/dfloat(nconf)
93    epot_avg = epot_sum/dfloat(nconf)
94    etot_avg = ekin_avg + epot_avg + etail
95    pkin_avg = pkin_sum/dfloat(nconf)
96    pvir_avg = pvir_sum/dfloat(nconf)
97    ptot_avg = pkin_avg + pvir_avg + ptail
```

Listing 5: Run averages of the energy and pressure contributions.

```
120   ! 6. Radial distribution function in reduced units.
121   open(15, file='g-leap.dat', status='unknown')
122   do j = 1, nhis
123      rr = delg*(dfloat(j) - 0.5d0)
124      vb = (dfloat(j)**3 - dfloat(j-1)**3)*delg**3
125      nid = (4.d0/3.d0)*pi*vb*rho
126      g(j) = g(j)/(dfloat(nconf)*dfloat(natoms)*nid)
127      write(15,*) rr, g(j)
128   end do
129   close(15)
```

Listing 6: Normalisation of $g(r)$ by the ideal-gas content of each shell.

```
183   epot = 0.d0
184   vir = 0.d0
185
186   ! atom-atom interactions
187   do is = 1, natoms-1
188      do js = is+1, natoms
189         call lj(is, js, r, boxlength, accel, rc, pot, virij, nhis, g, delg)
190         epot = epot + pot
191         vir = vir + virij
192      end do
193   end do
```

Listing 7: Pair virial summed inside `forces`.

```
230      virij = forcedist*rr2
231   end if
232
233   ! Histogram of pair distances up to L/2 (minimum image).
234   if (rr .lt. boxlength/2.d0) then
235      ig = int(rr/delg) + 1
236      if (ig .ge. 1 .and. ig .le. nhis) g(ig) = g(ig) + 2.d0
237   end if
```

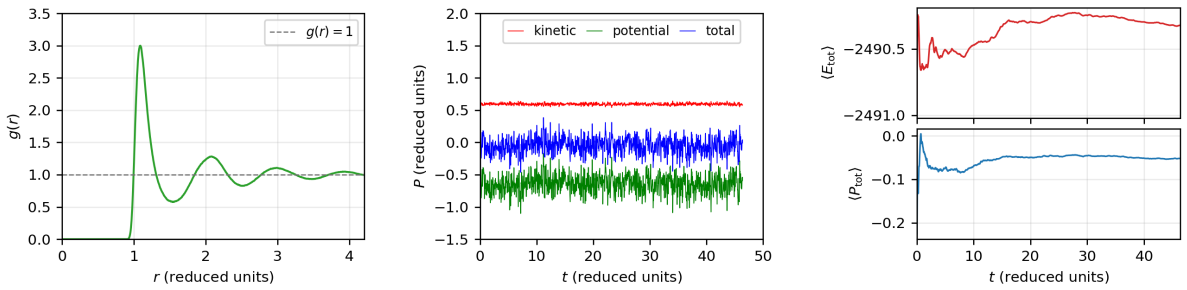
Listing 8: End of subroutine `lj`: virial of one pair inside the cutoff, and the $g(r)$ shell counted up to $L/2$.

2.3 Results

The same 10000-step trajectory as in Exercise 1 ($N = 500$, $\rho^* = 0.835$, $\langle T^* \rangle = 0.713$, $r_c = 2.5$) was used. $g(r)$ is accumulated in 1000 shells up to $L^*/2 = 4.21$ and normalised by the ideal-gas content of each shell. The pressure is $P = P_{\text{kin}} + P_{\text{vir}} + P_{\text{tail}}$ with $P_{\text{kin}} = 2E_{\text{kin}}/(3V)$ and $P_{\text{vir}} = \frac{1}{3V}\sum_{i<j} \vec{r}_{ij} \cdot \vec{f}_{ij}$. The tails assume $g(r) = 1$ beyond r_c:

$$E_{\text{tail}} = \frac{8\pi}{3}\rho N\left(\frac{1}{3r_c^9} - \frac{1}{r_c^3}\right) = -223.67, \qquad P_{\text{tail}} = \frac{16\pi}{3}\rho^2\left(\frac{2}{3r_c^9} - \frac{1}{r_c^3}\right) = -0.7465. \tag{2}$$

Figure 2 shows $g(r)$, the pressure contributions and the running means of the totals. Table 1 gives the mean of every contribution after increasing simulation lengths. The last row is the full run.



(a) RDF (b) Pressure (c) Convergence

Figure 2: Structure and thermodynamics of the Lennard-Jones liquid in reduced units. (a) Radial distribution function $g(r)$. (b) Kinetic, potential ($P_{\mathrm{vir}} + P_{\mathrm{tail}}$) and total pressure versus time. (c) Running means of E_{tot} and P_{tot} (tails included) from $t = 0$ to t.

Table 1: Mean energy and pressure contributions (reduced units) after n steps. $E_{\mathrm{tail}} = -223.67$ and $P_{\mathrm{tail}} = -0.7465$ are constant and are included in the totals.

n	t^*	$\langle E_{\mathrm{kin}} \rangle$	$\langle E_{\mathrm{pot}} \rangle$	$\langle E_{\mathrm{tot}} \rangle$	$\langle P_{\mathrm{kin}} \rangle$	$\langle P_{\mathrm{vir}} \rangle$	$\langle P_{\mathrm{tot}} \rangle$
1000	4.64	535.49	−2802.35	−2490.53	0.5965	0.0721	−0.0779
2500	11.59	534.82	−2801.60	−2490.45	0.5958	0.0922	−0.0585
5000	23.19	534.06	−2800.66	−2490.27	0.5949	0.1042	−0.0473
7500	34.78	533.35	−2799.94	−2490.26	0.5941	0.1061	−0.0462
10000	46.37	533.66	−2800.31	−2490.32	0.5945	0.1001	−0.0519

2.4 Discussion & Notes

- **RDF:** $g(r) = 0$ inside the repulsive core ($r \lesssim 0.93$) and then oscillates as in a dense liquid, matching the reference curve. The first peak is at $r = 1.089$ ($g = 3.00$), the first minimum at $r = 1.544$ ($g = 0.576$), and further peaks at $r = 2.079$ ($g = 1.28$) and $r = 2.985$ ($g = 1.11$). The oscillations die out and $g(L/2) = 0.993 \approx 1$: there is short-range order but no long-range order.

- **Energy:** E_{kin} and E_{pot} each fluctuate by $\sigma \approx 13$, in anti-phase, so their sum keeps the $\sigma = 0.537$ of Exercise 1. $\langle E_{\mathrm{tot}} \rangle / N = -4.98$ per particle, and the tail is 9% of E_{tot}.

- **Pressure:** $\langle P_{\mathrm{kin}} \rangle = 0.5945 \approx \rho^* \langle T^* \rangle = 0.596$. The repulsive virial 0.100 and the attractive tail -0.746 almost cancel it, so $P_{\mathrm{tot}} = -0.052$ fluctuates around zero ($\sigma_P = 0.120$), as in the reference figure. Without the tail P^* would be $+0.69$, so the tail correction is essential at $r_c = 2.5$.

- **Convergence:** $\langle E_{\mathrm{tot}} \rangle$ changes by only 0.2 (less than 0.01%) once a few thousand steps have been averaged. $\langle P_{\mathrm{tot}} \rangle$ converges more slowly: it is a small difference of large terms, and its instantaneous fluctuations are more than twice its mean. It settles after about 5000 steps ($t^* \approx 23$) to within ± 0.006. Block averaging the full run (10 blocks) gives statistical errors of 0.07 on E_{tot} and 0.007 on P_{tot}, so 10000 steps are enough for both.

3 Exercise 3

3.1 Question

STEP 3: Analysing the dynamics.

- Modify the code to calculate the velocity autocorrelation function (VACF) and the mean square displacement (MSD):

$$C(t) = \langle \vec{v}(t) \cdot \vec{v}(0) \rangle, \tag{3}$$

$$r^2(t) = \langle [\vec{r}(t) - \vec{r}(0)]^2 \rangle. \tag{4}$$

  Compute the VACF and the MSD for a run of 10000 steps. Plot the results.

- Write a code to calculate the spectral density (SDW) of the VACF:

$$S(\omega) = \int_0^\infty dt\, C(t) \cos(\omega t). \tag{5}$$

  Use a Simpson 3/8 rule or similar. Plot the results.

- Use Einstein's diffusion formula to obtain the self-diffusion coefficient D:

$$D = \lim_{t\to\infty} \frac{1}{6t} r^2(t). \tag{6}$$

3.2 Code

Only the lines added in this step are shown, with a few lines of the existing program kept where they mark the insertion point. The full program is listed in the Anex.

```
11    ! 1. Defining dimensions
12    dimension r(3,1000), vinf(3,1000), accel(3,1000), runw(3,1000), rold(3,1000)
13    dimension c(0:10000), r2(0:10000)
14    parameter (nhis = 1000)
15    dimension g(nhis)
16    dimension t(0:10000), D(10000), c_spectrum(0:1000)
17    ! Multiple time origins for VACF and MSD (lecture SAMPLE algorithm).
18    ! init=0 initialize, init=1 accumulate a sample, init=2 normalize.
19    integer :: ncor, it0, ntmax, t0max, ntel, t0
20    integer, allocatable :: time0(:), ntime(:)
21    double precision, allocatable :: vx0(:,:,:), x0(:,:,:), vacf(:), r2t(:)
```

Listing 9: Unwrapped-trajectory arrays `runw`, `rold`, and storage for the VACF, the MSD, the spectral density and the time-origin buffers.

```
51    ! 4. Change to reduced units
52    call reduced(natoms, r, vinf, boxlength, deltat, epsil, sigma, &
53                 mass, uvel)
54
55    ! runw holds positions without periodic wrapping, needed for the MSD.
56    runw = r
```

Listing 10: Unwrapped coordinates, initialised once the positions are in reduced units.

```
74    ! Correlation window of about 5 reduced time units: the VACF has
75    ! decayed and the MSD is already linear. ncor is a multiple of 6 so
76    ! the Simpson 3/8 rule has a complete set of panels. A new time
77    ! origin is taken every it0 samples; t0max keeps every origin still
78    ! inside the window. ntmax is tmax in the lecture listing (1-based,
79    ! with C(0) stored in vacf(1)).
80    it0 = 5
```

```
81    ncor = nint(5.d0/deltat)
82    ncor = ncor - mod(ncor, 6)
83    if (ncor < 6) ncor = 6
84    if (ncor > nconf) ncor = nconf - mod(nconf, 6)
85    ntmax = ncor + 2
86    t0max = ncor/it0 + 3
87    allocate(time0(t0max), ntime(ntmax))
88    allocate(vx0(3, natoms, t0max), x0(3, natoms, t0max))
89    allocate(vacf(ntmax), r2t(ntmax))
90    call sample(0)
```

Listing 11: Correlation window and circular buffer of time origins.

```
99        call forces(natoms, r, boxlength, accel, rc, epot, vir, nhis, g, delg)
100       rold = r
101       call velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, boxlength)
102
103       ! Unfold this step (SAMPLE does not apply periodic wrapping to r2t).
104       do j = 1, natoms
105          do l = 1, 3
106             dr = r(l,j) - rold(l,j)
107             runw(l,j) = runw(l,j) + dr - boxlength*dnint(dr/boxlength)
108          end do
109       end do
110       call sample(1)
```

Listing 12: Each leap-frog step is unfolded before `sample` accumulates $C(t)$ and $r^2(t)$.

```
169   ! 7. Normalize VACF and MSD (SAMPLE init=2) and copy to 0-based arrays.
170   call sample(2)
171   do i = 0, ncor
172      c(i) = vacf(i+1)
173      r2(i) = r2t(i+1)
174      t(i) = i*deltat
175   end do
176   print *, 'VACF and MSD averaged over time origins'
177   print *, '  origins at t = 0     =', ntime(1)
178   print *, '  origins at t = tmax  =', ntime(ncor+1)
179   print *, '  C(0) =', c(0)
180
181   open(16, file='c-leap.dat', status='unknown')
182   open(17, file='r2-leap.dat', status='unknown')
183   do i = 0, ncor
184      write(16,*) t(i), c(i)
185      write(17,*) t(i), r2(i)
186   end do
187   close(16)
188   close(17)
189
190   ! 8. Spectral density of the averaged VACF, Simpson 3/8 rule.
191   !    Frequencies w = 0, 1, ..., 100. Only those are written: the
192   !    previous loop stopped at 100 and the rest of the file was unset.
193   call spectrum(c(0:ncor), t(0:ncor), c_spectrum)
194   open(18, file='c_spectrum-leap.dat', status='unknown')
195   do i = 0, 100
196      write(18,*) dfloat(i), c_spectrum(i)
197   end do
198   close(18)
```

Listing 13: Normalisation and output of the VACF and the MSD, and spectral density of $C(t)$.

```
208   ! 10. Least-squares linear fit r2(t) = 6*D*t + b over the diffusive regime.
209   !     The first quarter of the window (ballistic regime) is discarded.
```

```
!     yint is used instead of "intercept": implicit typing would make it integer.
ifit = ncor/4
npts = ncor - ifit + 1
sx = 0.d0
sy = 0.d0
sxx = 0.d0
sxy = 0.d0
do i = ifit, ncor
   sx = sx + t(i)
   sy = sy + r2(i)
   sxx = sxx + t(i)*t(i)
   sxy = sxy + t(i)*r2(i)
end do
slope = (dfloat(npts)*sxy - sx*sy)/(dfloat(npts)*sxx - sx*sx)
yint = (sy - slope*sx)/dfloat(npts)
d_einstein = slope/6.d0
tmin = t(ifit)
tmax = t(ncor)
```

Listing 14: Einstein self-diffusion coefficient from a linear fit of $r^2(t)$ in the diffusive regime.

```
!*************************************************************
!     SAMPLE: VACF and MSD with multiple time origins
!     init=0 initialize counters
!     init=1 accumulate a sample of data
!     init=2 average the results
!     Positions in x0 / runw are unwrapped (PBC not assumed).
!*************************************************************

subroutine sample(init)
  implicit none
  integer, intent(in) :: init
  integer :: i, j, l, tt0, torg, delt
  double precision :: dx

  if (init == 0) then
     ntel = 0
     t0 = 0
     do i = 1, ntmax
        ntime(i) = 0
        vacf(i) = 0.d0
        r2t(i) = 0.d0
     end do
  else if (init == 1) then
     ntel = ntel + 1
     if (mod(ntel, it0) == 0) then
        t0 = t0 + 1
        tt0 = mod(t0-1, t0max) + 1
        time0(tt0) = ntel
        do j = 1, natoms
           do l = 1, 3
              x0(l,j,tt0) = runw(l,j)
              vx0(l,j,tt0) = vinf(l,j)
           end do
        end do
     end if
     do torg = 1, min(t0, t0max)
        delt = ntel - time0(torg) + 1
        if (delt < ntmax) then
           ntime(delt) = ntime(delt) + 1
           do j = 1, natoms
              do l = 1, 3
                 vacf(delt) = vacf(delt) + vinf(l,j)*vx0(l,j,torg)
                 dx = runw(l,j) - x0(l,j,torg)
```

```
                  r2t(delt) = r2t(delt) + dx*dx
               end do
            end do
         end if
      end do
   else if (init == 2) then
      do i = 1, ntmax
         if (ntime(i) > 0) then
            vacf(i) = vacf(i)/(dfloat(natoms)*dfloat(ntime(i)))
            r2t(i) = r2t(i)/(dfloat(natoms)*dfloat(ntime(i)))
         end if
      end do
   end if
end subroutine sample
```

Listing 15: Internal subroutine `sample`: VACF and MSD with multiple time origins.

```
!*********************************************************
!     spectral density, composite Simpson 3/8 rule
!     S(w) = integral C(t) cos(w t) dt, w = 0, 1, ..., 100
!*********************************************************

subroutine spectrum(C, t, result)
  implicit none
  double precision, dimension(:), intent(in) :: C, t
  double precision, dimension(0:), intent(out) :: result
  integer :: i, w, n, panel
  double precision :: h, s
  double precision, allocatable :: f(:)

  n = int(0.25*size(t))
  h = t(2) - t(1)
  allocate(f(n))
  result = 0.d0

  do w = 0, 100
     do i = 1, n
        f(i) = C(i)*cos(dble(w)*t(i))
     end do

     ! Composite Simpson 3/8: panels come in groups of three.
     s = f(1) + f(n)
     do i = 2, n - 1
        if (mod(i - 1, 3) == 0) then
           s = s + 2.d0*f(i)
        else
           s = s + 3.d0*f(i)
        end if
     end do
     result(w) = 3.d0*h/8.d0*s
  end do

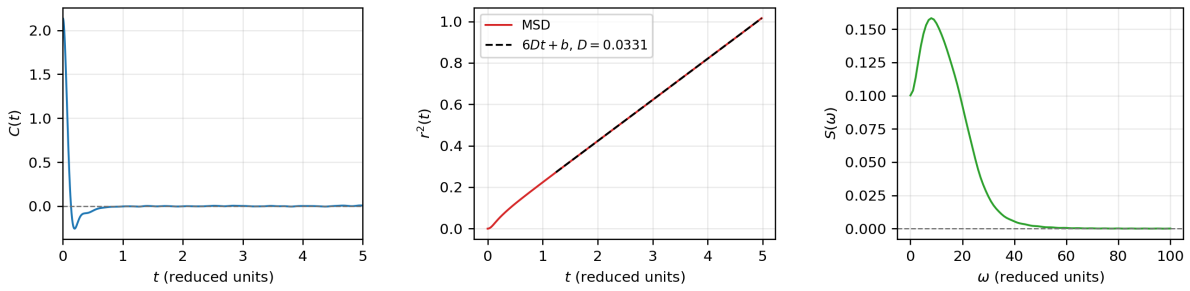
  deallocate(f)
end subroutine spectrum
```

Listing 16: Internal subroutine `spectrum`: composite Simpson 3/8 cosine transform of the VACF.

3.3 Results

On the same 10000-step trajectory ($N = 500$, $\rho^* = 0.835$, $\langle T^* \rangle = 0.713$), $C(t)$ and $r^2(t)$ were averaged over particles and time origins in a window of 1074 steps ($t^* \leq 4.98$). Figure 3 shows the VACF, the MSD with its Einstein fit, and the spectral density $S(\omega)$. The fit gives a slope

of 0.1988 and an intercept of 0.025, so $D^* = 0.0331$, i.e. $D = D^*\sigma^2/\tau = 0.178$ Å^2 ps^{-1} $= 1.78 \times 10^{-5}$ cm^2 s^{-1}.



(a) VACF (b) MSD (c) SDW

Figure 3: Dynamics of the Lennard-Jones liquid in reduced units. (a) Velocity autocorrelation $C(t)$. (b) Mean square displacement $r^2(t)$; the dashed line is the fit $r^2 = 6Dt + b$ on $t^* \in [1.24, 4.98]$. (c) Spectral density $S(\omega)$ of the VACF, Simpson 3/8 rule, $\omega = 0, 1, \ldots, 100$.

3.4 Discussion & Notes

- **Method and setup:** Following the lecture SAMPLE scheme, a new time origin is stored every 5 steps in a circular buffer. The MSD uses unwrapped positions: each leap-frog displacement is unfolded with the minimum image, so $r^2(t)$ is not folded back by the periodic box. The window $n_{\text{cor}} = 1074$ is a multiple of 6, so Simpson's 3/8 rule has complete panels, and it is long enough for $C(t)$ to vanish and for $r^2(t)$ to become linear. The integral (5) is evaluated on $t^* \leq 1.24$ (first quarter of the window), beyond which $C(t) \approx 0$. Following (6), D is one sixth of the slope of $r^2(t)$ fitted over the last three quarters of the window, which discards the ballistic rise.

- **Results vs expectations:**

  - *VACF:* $C(0) = 2.138$ agrees with the equipartition value $\langle v^2 \rangle = 3\langle T^* \rangle = 2.139$ ($m = 1$). $C(t)$ decays quickly, crosses zero at $t^* = 0.139$ and reaches a minimum $C = -0.256$ at $t^* = 0.195$. This negative lobe is the signature of a dense liquid: atoms rebound from the cage of first neighbours (the RDF peak at $r = 1.09$ in Exercise 2). Beyond $t^* \approx 0.64$, $|C| < 0.02$ and $C(t)$ only fluctuates around zero.
  - *MSD:* $r^2(t)$ follows the ballistic law $\langle v^2 \rangle t^2$ only for $t^* \lesssim 0.05$ ($r^2 = 0.0053$ against 0.0056), bends as collisions set in, and is linear for $t^* \gtrsim 1$. It reaches $r^2 = 1.016$ at $t^* = 4.98$, so each atom moves about one σ in the window.
  - *SDW:* $S(\omega)$ peaks at $\omega = 8$ ($S = 0.158$), the characteristic frequency of an atom rattling in its cage and the counterpart of the negative VACF lobe. It falls to 5.4×10^{-3} at $\omega = 40$ and 5×10^{-5} at $\omega = 100$, so no motion is faster than the collision time. $S(0) = 0.100$ is finite, as it must be for a fluid (a solid has $S(0) = 0$), and $S(0)/3 = 0.0334$ matches the Einstein D^* to 1%.
  - *Self-diffusion:* $D = 1.78 \times 10^{-5}$ cm^2 s^{-1} is of the order expected for simple liquids near their triple point.

- **Conclusion:** A window of 5 reduced time units suffices: the VACF has died out and the MSD is linear. The cage-backscattering lobe of $C(t)$, the peak of $S(\omega)$ at $\omega = 8$ and $D^* = 0.033$ all describe a dense Lennard-Jones liquid, consistent with the RDF of Exercise 2.

Anex

The program listed here is the full source used throughout the practice. It is the leap-frog integrator supplied for Exercise 1, together with the radial distribution function, the virial pressure and the tail corrections added in Exercise 2, and the velocity autocorrelation, mean square displacement, spectral density and self-diffusion coefficient added in Exercise 3. Each exercise section quotes only the lines introduced in that step.

```
!***************************************************************************
!          Molecular Dynamics code to simulate at NVE ensemble
!          a system of N atoms of Ar inside a cubic box, interacting
!          through a truncated Lennard-Jones force field at 2.5 sigma.
!          Leap-frog Verlet integration algorithm.
!***************************************************************************
program leapfroglj
   implicit double precision(a-h,o-z)
   double precision mass, nid

   ! 1. Defining dimensions
   dimension r(3,1000), vinf(3,1000), accel(3,1000), runw(3,1000), rold(3,1000)
   dimension c(0:10000), r2(0:10000)
   parameter (nhis = 1000)
   dimension g(nhis)
   dimension t(0:10000), D(10000), c_spectrum(0:1000)
   ! Multiple time origins for VACF and MSD (lecture SAMPLE algorithm).
   ! init=0 initialize, init=1 accumulate a sample, init=2 normalize.
   integer :: ncor, it0, ntmax, t0max, ntel, t0
   integer, allocatable :: time0(:), ntime(:)
   double precision, allocatable :: vx0(:,:,:), x0(:,:,:), vacf(:), r2t(:)

   INCLUDE '../../chdir_to_code.inc'

   ! 2. Reading data and computing related quantities
   open(1, file='../data/leap-lj.data', status='old')
   read(1,*) nconf
   read(1,*) natoms
   read(1,*) mass
   read(1,*) sigma, epsil
   read(1,*) deltat
   close(1)

   nf = 3*natoms - 3  ! number of degrees of freedom
   rc = 2.5d0         ! cutoff radius in units of sigma

   ! 3. Reading initial configuration (positions, velocities) in A and A/ps
   open(2, file='../data/leap-conf.data', status='old')
   do is = 1, natoms
      read(2,*) (r(l,is), l=1,3)
      read(2,*) (vinf(l,is), l=1,3)
   end do
   read(2,*) boxlength
   close(2)

   ! Opening files to write results
   open(3, file='energy-leap.dat', status='unknown')
   open(4, file='temp-leap.dat', status='unknown')
   open(8, file='thermo-leap.dat', status='unknown')

   ! 4. Change to reduced units
   call reduced(natoms, r, vinf, boxlength, deltat, epsil, sigma, &
                mass, uvel)

   ! runw holds positions without periodic wrapping, needed for the MSD.
   runw = r

```

```
58      ! 5. Start the loop to generate new configurations
59      pi = 4.d0*datan(1.d0)
60      delg = boxlength/(2.d0*dfloat(nhis))
61      rho = dfloat(natoms)/boxlength**3
62      etail = (8.d0*pi/3.d0)*rho*dfloat(natoms)* &
63              (1.d0/(3.d0*rc**9) - 1.d0/rc**3)
64      ptail = (16.d0*pi/3.d0)*rho**2* &
65              (2.d0/(3.d0*rc**9) - 1.d0/rc**3)
66      do j = 1, nhis
67         g(j) = 0.d0
68      end do
69      ekin_sum = 0.d0
70      epot_sum = 0.d0
71      pkin_sum = 0.d0
72      pvir_sum = 0.d0
73
74      ! Correlation window of about 5 reduced time units: the VACF has
75      ! decayed and the MSD is already linear. ncor is a multiple of 6 so
76      ! the Simpson 3/8 rule has a complete set of panels. A new time
77      ! origin is taken every it0 samples; t0max keeps every origin still
78      ! inside the window. ntmax is tmax in the lecture listing (1-based,
79      ! with C(0) stored in vacf(1)).
80      it0 = 5
81      ncor = nint(5.d0/deltat)
82      ncor = ncor - mod(ncor, 6)
83      if (ncor < 6) ncor = 6
84      if (ncor > nconf) ncor = nconf - mod(nconf, 6)
85      ntmax = ncor + 2
86      t0max = ncor/it0 + 3
87      allocate(time0(t0max), ntime(ntmax))
88      allocate(vx0(3, natoms, t0max), x0(3, natoms, t0max))
89      allocate(vacf(ntmax), r2t(ntmax))
90      call sample(0)
91
92      write(8,'(a)') '# t etot pkin pvir ptail ptot'
93
94      print *, 'Iniciando simulacion MD (', nconf, ' pasos)...'
95      do i = 1, nconf
96         if (mod(i, 1000) == 0) then
97            print *, '  -> Completado paso ', i, ' de ', nconf
98         end if
99         call forces(natoms, r, boxlength, accel, rc, epot, vir, nhis, g, delg)
100        rold = r
101        call velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, boxlength)
102
103        ! Unfold this step (SAMPLE does not apply periodic wrapping to r2t).
104        do j = 1, natoms
105           do l = 1, 3
106              dr = r(l,j) - rold(l,j)
107              runw(l,j) = runw(l,j) + dr - boxlength*dnint(dr/boxlength)
108           end do
109        end do
110        call sample(1)
111
112        ! Conserved NVE energy (no tail). Thermodynamic totals include it.
113        etot = ecin + epot
114        etot_corr = etot + etail
115        pkin = 2.d0*ecin/(3.d0*boxlength**3)
116        pvir = vir/(3.d0*boxlength**3)
117        ptot = pkin + pvir + ptail
118        ekin_sum = ekin_sum + ecin
119        epot_sum = epot_sum + epot
120        pkin_sum = pkin_sum + pkin
```

```
121         pvir_sum = pvir_sum + pvir
122         write(3,*) i*deltat, etot
123         write(4,*) i*deltat, temp
124         write(8,*) i*deltat, etot_corr, pkin, pvir, ptail, ptot
125      end do
126      close(3)
127      close(4)
128      close(8)
129
130      ekin_avg = ekin_sum/dfloat(nconf)
131      epot_avg = epot_sum/dfloat(nconf)
132      etot_avg = ekin_avg + epot_avg + etail
133      pkin_avg = pkin_sum/dfloat(nconf)
134      pvir_avg = pvir_sum/dfloat(nconf)
135      ptot_avg = pkin_avg + pvir_avg + ptail
136
137      open(9, file='averages-leap.dat', status='unknown')
138      write(9,*) 'nconf', nconf
139      write(9,*) 'ekin', ekin_avg
140      write(9,*) 'epot', epot_avg
141      write(9,*) 'etail', etail
142      write(9,*) 'etot', etot_avg
143      write(9,*) 'pkin', pkin_avg
144      write(9,*) 'pvir', pvir_avg
145      write(9,*) 'ptail', ptail
146      write(9,*) 'ptot', ptot_avg
147      close(9)
148      print *, 'Mean values in reduced units'
149      print *, '  Ekin  =', ekin_avg
150      print *, '  Epot  =', epot_avg
151      print *, '  Etail =', etail
152      print *, '  Etot  =', etot_avg
153      print *, '  Pkin  =', pkin_avg
154      print *, '  Pvir  =', pvir_avg
155      print *, '  Ptail =', ptail
156      print *, '  Ptot  =', ptot_avg
157
158      ! 6. Radial distribution function in reduced units.
159      open(15, file='g-leap.dat', status='unknown')
160      do j = 1, nhis
161         rr = delg*(dfloat(j) - 0.5d0)
162         vb = (dfloat(j)**3 - dfloat(j-1)**3)*delg**3
163         nid = (4.d0/3.d0)*pi*vb*rho
164         g(j) = g(j)/(dfloat(nconf)*dfloat(natoms)*nid)
165         write(15,*) rr, g(j)
166      end do
167      close(15)
168
169      ! 7. Normalize VACF and MSD (SAMPLE init=2) and copy to 0-based arrays.
170      call sample(2)
171      do i = 0, ncor
172         c(i) = vacf(i+1)
173         r2(i) = r2t(i+1)
174         t(i) = i*deltat
175      end do
176      print *, 'VACF and MSD averaged over time origins'
177      print *, '  origins at t = 0     =', ntime(1)
178      print *, '  origins at t = tmax  =', ntime(ncor+1)
179      print *, '  C(0) =', c(0)
180
181      open(16, file='c-leap.dat', status='unknown')
182      open(17, file='r2-leap.dat', status='unknown')
183      do i = 0, ncor
```

```
184         write(16,*) t(i), c(i)
185         write(17,*) t(i), r2(i)
186      end do
187      close(16)
188      close(17)
189
190      ! 8. Spectral density of the averaged VACF, Simpson 3/8 rule.
191      !    Frequencies w = 0, 1, ..., 100. Only those are written: the
192      !    previous loop stopped at 100 and the rest of the file was unset.
193      call spectrum(c(0:ncor), t(0:ncor), c_spectrum)
194      open(18, file='c_spectrum-leap.dat', status='unknown')
195      do i = 0, 100
196         write(18,*) dfloat(i), c_spectrum(i)
197      end do
198      close(18)
199
200      ! 9. Instantaneous diffusion coefficient from the mean square displacement
201      open(19, file='D-leap.dat', status='unknown')
202      do i = 1, ncor
203         D(i) = r2(i)/(6.d0*t(i))
204         write(19,*) t(i), D(i)
205      end do
206      close(19)
207
208      ! 10. Least-squares linear fit r2(t) = 6*D*t + b over the diffusive regime.
209      !     The first quarter of the window (ballistic regime) is discarded.
210      !     yint is used instead of "intercept": implicit typing would make it integer.
211      ifit = ncor/4
212      npts = ncor - ifit + 1
213      sx = 0.d0
214      sy = 0.d0
215      sxx = 0.d0
216      sxy = 0.d0
217      do i = ifit, ncor
218         sx = sx + t(i)
219         sy = sy + r2(i)
220         sxx = sxx + t(i)*t(i)
221         sxy = sxy + t(i)*r2(i)
222      end do
223      slope = (dfloat(npts)*sxy - sx*sy)/(dfloat(npts)*sxx - sx*sx)
224      yint = (sy - slope*sx)/dfloat(npts)
225      d_einstein = slope/6.d0
226      tmin = t(ifit)
227      tmax = t(ncor)
228
229      ! Green-Kubo: D = (1/3) * integral of C(t) from 0 to the correlation window.
230      d_gk = deltat*(sum(c(0:ncor)) - 0.5d0*c(0) - 0.5d0*c(ncor))/3.d0
231
232      open(20, file='diffusion-leap.dat', status='unknown')
233      write(20,'(a)') '# D_einstein slope intercept tmin tmax D_greenkubo'
234      write(20,*) d_einstein, slope, yint, tmin, tmax, d_gk
235      close(20)
236      print *, 'Diffusion coefficient (reduced units)'
237      print *, '  D (Einstein fit) =', d_einstein
238      print *, '  D (Green-Kubo)   =', d_gk
239
240      ! 7. Saving last configuration in A and A/ps
241      open(11, file='newconf.data', status='unknown')
242      do is = 1, natoms
243         write(11,*) (r(l,is)*sigma, l=1,3)
244         write(11,*) (vinf(l,is)*uvel, l=1,3)
245      end do
246      write(11,*) boxlength*sigma
```

```
  close(11)

  stop

contains

!*********************************************************
!     SAMPLE: VACF and MSD with multiple time origins
!     init=0 initialize counters
!     init=1 accumulate a sample of data
!     init=2 average the results
!     Positions in x0 / runw are unwrapped (PBC not assumed).
!*********************************************************

subroutine sample(init)
  implicit none
  integer, intent(in) :: init
  integer :: i, j, l, tt0, torg, delt
  double precision :: dx

  if (init == 0) then
     ntel = 0
     t0 = 0
     do i = 1, ntmax
        ntime(i) = 0
        vacf(i) = 0.d0
        r2t(i) = 0.d0
     end do
  else if (init == 1) then
     ntel = ntel + 1
     if (mod(ntel, it0) == 0) then
        t0 = t0 + 1
        tt0 = mod(t0-1, t0max) + 1
        time0(tt0) = ntel
        do j = 1, natoms
           do l = 1, 3
              x0(l,j,tt0) = runw(l,j)
              vx0(l,j,tt0) = vinf(l,j)
           end do
        end do
     end if
     do torg = 1, min(t0, t0max)
        delt = ntel - time0(torg) + 1
        if (delt < ntmax) then
           ntime(delt) = ntime(delt) + 1
           do j = 1, natoms
              do l = 1, 3
                 vacf(delt) = vacf(delt) + vinf(l,j)*vx0(l,j,torg)
                 dx = runw(l,j) - x0(l,j,torg)
                 r2t(delt) = r2t(delt) + dx*dx
              end do
           end do
        end if
     end do
  else if (init == 2) then
     do i = 1, ntmax
        if (ntime(i) > 0) then
           vacf(i) = vacf(i)/(dfloat(natoms)*dfloat(ntime(i)))
           r2t(i) = r2t(i)/(dfloat(natoms)*dfloat(ntime(i)))
        end if
     end do
  end if
end subroutine sample
```

```
310
311  !*********************************************************
312  !     spectral density, composite Simpson 3/8 rule
313  !     S(w) = integral C(t) cos(w t) dt, w = 0, 1, ..., 100
314  !*********************************************************
315
316  subroutine spectrum(C, t, result)
317    implicit none
318    double precision, dimension(:), intent(in) :: C, t
319    double precision, dimension(0:), intent(out) :: result
320    integer :: i, w, n, panel
321    double precision :: h, s
322    double precision, allocatable :: f(:)
323
324    n = int(0.25*size(t))
325    h = t(2) - t(1)
326    allocate(f(n))
327    result = 0.d0
328
329    do w = 0, 100
330       do i = 1, n
331          f(i) = C(i)*cos(dble(w)*t(i))
332       end do
333
334       ! Composite Simpson 3/8: panels come in groups of three.
335       s = f(1) + f(n)
336       do i = 2, n - 1
337          if (mod(i - 1, 3) == 0) then
338             s = s + 2.d0*f(i)
339          else
340             s = s + 3.d0*f(i)
341          end if
342       end do
343       result(w) = 3.d0*h/8.d0*s
344    end do
345
346    deallocate(f)
347  end subroutine spectrum
348
349  end program leapfroglj
350
351  !*********************************************************
352  !              subroutine reduced
353  !*********************************************************
354
355  subroutine reduced(natoms, r, vinf, boxlength, deltat, &
356                     epsil, sigma, mass, uvel)
357    implicit double precision(a-h,o-z)
358    double precision mass
359    dimension r(3,1000), vinf(3,1000)
360
361    rgas = 8.314472673d0  ! J/(mol*K)
362    utime = sigma*dsqrt(mass/epsil)*dsqrt(10.d0/rgas)
363    uvel = sigma/utime    ! unit of velocity, expressed in A/ps
364
365    boxlength = boxlength/sigma
366    deltat = deltat/utime
367    do is = 1, natoms
368       do l = 1, 3
369          r(l,is) = r(l,is)/sigma
370          vinf(l,is) = vinf(l,is)/uvel
371       end do
372    end do
```

```
373
374     return
375   end subroutine reduced
376
377   !*************************************************************
378   !              subroutine forces
379   !*************************************************************
380
381   subroutine forces(natoms, r, boxlength, accel, rc, epot, vir, nhis, g, delg)
382     implicit double precision(a-h,o-z)
383     dimension r(3,1000), accel(3,1000)
384     dimension g(nhis)
385
386     do is = 1, natoms
387        do l = 1, 3
388           accel(l,is) = 0.d0  ! sets accelerations to 0
389        end do
390     end do
391     epot = 0.d0
392     vir = 0.d0
393
394     ! atom-atom interactions
395     do is = 1, natoms-1
396        do js = is+1, natoms
397           call lj(is, js, r, boxlength, accel, rc, pot, virij, nhis, g, delg)
398           epot = epot + pot
399           vir = vir + virij
400        end do
401     end do
402
403     return
404   end subroutine forces
405
406   !*************************************************************
407   !              subroutine Lennard-Jones
408   !*************************************************************
409
410   subroutine lj(is, js, r, boxlength, accel, rc, pot, virij, nhis, g, delg)
411     implicit double precision(a-h,o-z)
412     dimension r(3,1000), accel(3,1000)
413     dimension rij(3)
414     dimension g(nhis)
415
416     rr2 = 0.d0
417     pot = 0.d0
418     virij = 0.d0
419
420     do l = 1, 3
421        rijl = r(l,js) - r(l,is)
422        rij(l) = rijl - boxlength*dnint(rijl/boxlength)
423        rr2 = rr2 + rij(l)*rij(l)
424     end do
425
426     rr = dsqrt(rr2)
427
428     if (rr .lt. rc) then
429        ynvrr2 = 1.d0/rr2
430        ynvrr6 = ynvrr2*ynvrr2*ynvrr2
431        ynvrr12 = ynvrr6*ynvrr6
432        forcedist = 24.d0*(2.d0*ynvrr12 - ynvrr6)*ynvrr2
433        pot = 4.d0*(ynvrr12 - ynvrr6)
434        do l = 1, 3
435           accel(l,is) = accel(l,is) - forcedist*rij(l)
```

```
436           accel(l,js) = accel(l,js) + forcedist*rij(l)
437        end do
438        virij = forcedist*rr2
439     end if
440
441     ! Histogram of pair distances up to L/2 (minimum image).
442     if (rr .lt. boxlength/2.d0) then
443        ig = int(rr/delg) + 1
444        if (ig .ge. 1 .and. ig .le. nhis) g(ig) = g(ig) + 2.d0
445     end if
446
447     return
448   end subroutine lj
449
450   !*********************************************************
451   !              subroutine velpos
452   !*********************************************************
453
454   subroutine velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, &
455                     boxlength)
456     implicit double precision(a-h,o-z)
457     dimension vinf(3,1000), accel(3,1000), r(3,1000)
458
459     ecin = 0.d0
460     do is = 1, natoms
461        v2 = 0.d0
462        do l = 1, 3
463           vsup = vinf(l,is) + accel(l,is)*deltat
464           r(l,is) = r(l,is) + vsup*deltat
465           v = (vsup + vinf(l,is))/2.d0
466           v2 = v2 + v*v
467           vinf(l,is) = vsup
468        end do
469        ecin = ecin + 0.5d0*v2
470     end do
471     temp = 2.d0*ecin/dfloat(nf)
472
473     ! Applying periodic boundary conditions
474     do is = 1, natoms
475        do l = 1, 3
476           if (r(l,is) .lt. 0.d0) r(l,is) = r(l,is) + boxlength
477           if (r(l,is) .gt. boxlength) r(l,is) = r(l,is) - boxlength
478        end do
479     end do
480
481     return
482   end subroutine velpos
```

Listing 17: Full leap-frog program used for the three exercises.